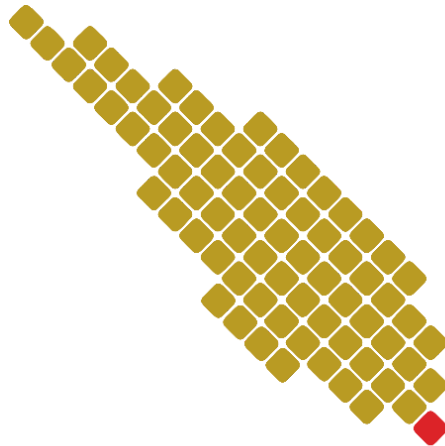


OpenFlights Route Finder: Optimasi Pencarian Jalur Penerbangan Menggunakan Queue dan Breadth First Search

Tugas Besar Struktur Data

Kelompok 2 RC:

1. Favian Arkaanda(124450021)
2. Riska Erlis Dayu Tiara(124450022)
3. Farhanah Hadaya Fatin (124450026)
4. M. Razan Maulana Pratama(124450031)
5. Hani Qurrota Aini(124450020)
6. Charrlindah(124450037)



**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN
2026**

DAFTAR ISI

DAFTAR ISI.....	1
DAFTAR GAMBAR.....	2
BAB 1	
PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Rumusan Masalah.....	3
1.3 Deskripsi Data.....	4
BAB II	
Metode dan Rancangan Sistem.....	5
2.1 Gambaran Umum Metode yang Digunakan.....	5
2.2 Flowchart Sistem.....	6
2.3 Dependency dan Library yang Digunakan.....	6
2.4 Daftar Fungsionalitas.....	7
BAB III	
HASIL DAN PEMBAHASAN.....	9
3.1 Hasil Implementasi Sistem.....	9
3.2 Hasil Pengujian Pencarian Rute Terpendek.....	9
3.3 Analisis Jangkauan Bandara.....	11
3.4 Hasil Pengecekan Konektivitas.....	12
3.5 Pembahasan Implementasi BFS dan Queue.....	12
BAB IV	
KESIMPULAN.....	14
DAFTAR PUSTAKA.....	15

DAFTAR GAMBAR

Gambar 2.1 Flowchart Sistem.....	6
Gambar 3.1 Output proses pemuatan data dan pembangunan graph.....	9
Gambar 3.2 Hasil pencarian rute KZN menuju LED.....	10
Gambar 3.3 Hasil pencarian rute SIN menuju BKK.....	10
Gambar 3.4 Hasil pencarian rute CEK menuju NBC dengan transit.....	11
Gambar 3.5 Analisis jangkauan bandara KZN.....	11
Gambar 3.6 Analisis jangkauan bandara SIN.....	12

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam era globalisasi yang terus berkembang, transportasi udara memegang peranan penting sebagai penghubung antarkota dan antarnegara di seluruh dunia. Jutaan penumpang setiap harinya bergantung pada jaringan penerbangan yang kompleks untuk berpindah dari satu tempat ke tempat lain, baik untuk keperluan bisnis, wisata, maupun keperluan pribadi lainnya. Jaringan penerbangan global terdiri dari ribuan bandara yang saling terhubung melalui rute-rute penerbangan yang dioperasikan oleh berbagai maskapai penerbangan di seluruh penjuru dunia.

Kompleksitas jaringan penerbangan ini menimbulkan tantangan tersendiri, khususnya dalam hal pencarian rute penerbangan yang paling efisien. Penumpang maupun sistem pemesanan tiket perlu mengetahui jalur tercepat atau jalur dengan transit paling sedikit untuk mencapai bandara tujuan. Masalah ini secara alami dapat dimodelkan sebagai permasalahan pencarian jalur terpendek pada sebuah graf, di mana setiap bandara direpresentasikan sebagai simpul (node) dan setiap rute penerbangan direpresentasikan sebagai sisi (edge) yang menghubungkan dua simpul.

Salah satu algoritma yang paling tepat untuk menyelesaikan permasalahan pencarian jalur terpendek berdasarkan jumlah hop atau transit minimum adalah Breadth-First Search (BFS). Algoritma BFS menelusuri graf secara berlapis dari simpul asal, sehingga jalur yang ditemukan pertama kali dijamin merupakan jalur dengan jumlah edge terkecil. Dalam konteks jaringan penerbangan, hal ini berarti BFS dapat menemukan rute dengan jumlah transit paling sedikit antara dua bandara. Oleh karena itu, proyek tugas besar ini mengimplementasikan algoritma BFS dengan memanfaatkan struktur data Queue untuk membangun sistem pencarian rute penerbangan terpendek berbasis data nyata.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah sebagai berikut.

1. Bagaimana cara merepresentasikan jaringan penerbangan global yang terdiri dari ribuan bandara dan puluhan ribu rute ke dalam struktur data graf yang efisien?
2. Bagaimana mengimplementasikan algoritma BFS menggunakan struktur data Queue untuk menemukan rute penerbangan terpendek dengan jumlah transit minimum antara dua bandara yang ditentukan?
3. Bagaimana menganalisis jangkauan konektivitas suatu bandara terhadap bandara-bandara lain berdasarkan jumlah level transit yang diperlukan?

4. Bagaimana memverifikasi apakah dua bandara tertentu saling terhubung dalam jaringan penerbangan global?

1.3 Deskripsi Data

Data yang digunakan dalam proyek ini bersumber dari dataset rute penerbangan internasional yang disimpan dalam file `routes.csv`. Dataset ini merupakan data nyata yang mencakup informasi rute penerbangan dari berbagai maskapai di seluruh dunia.

Dataset tersebut memiliki total 67.663 rute penerbangan yang mencakup 3.425 bandara dari berbagai negara di seluruh dunia. Setiap baris data merepresentasikan satu rute penerbangan dengan atribut-atribut sebagai berikut: kode maskapai penerbangan (airline) beserta ID-nya, kode IATA bandara asal (source airport) beserta ID-nya, kode IATA bandara tujuan (destination airport) beserta ID-nya, jumlah perhentian antara (stops), serta jenis pesawat yang digunakan (equipment).

Data rute tersebut bersifat berarah (directed), artinya rute dari bandara A ke bandara B tidak secara otomatis berarti terdapat rute dari B ke A. Graf yang dibangun dari data ini merupakan directed graph dengan 3.425 node dan puluhan ribu edge. Setelah data dimuat dan graf dibangun, dilakukan sejumlah pengujian menggunakan delapan use case yang mencakup berbagai skenario penerbangan, mulai dari rute domestik Rusia (KZN ke LED), rute Asia Tenggara (SIN ke BKK), rute internasional Asia (MNL ke KIX), rute dengan transit (CEK ke NBC, ACH ke MUC, FLL ke PNS), hingga analisis jangkauan per level dari bandara KZN (Kazan) dan SIN (Singapura).

BAB II

Metode dan Rancangan Sistem

2.1 Gambaran Umum Metode yang Digunakan

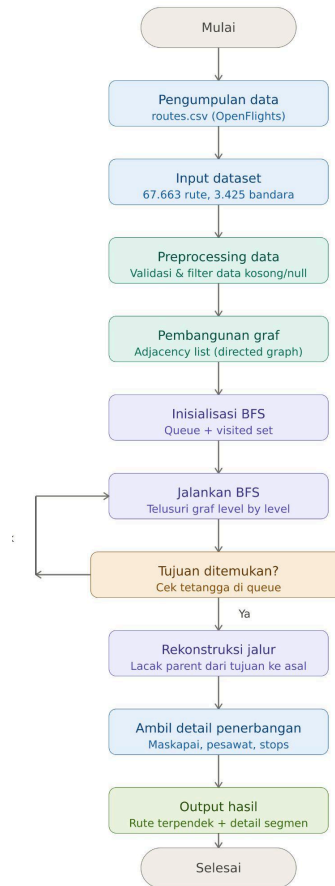
Sistem pencarian rute penerbangan terpendek ini dibangun menggunakan pendekatan pemodelan graf (graph modeling) yang dikombinasikan dengan algoritma Breadth-First Search (BFS). Berdasarkan Cormen et al. [1], algoritma BFS merupakan salah satu algoritma penelusuran graf yang bekerja secara berlapis dari simpul asal, sehingga jalur pertama yang ditemukan menuju simpul tujuan dijamin merupakan jalur dengan jumlah edge terkecil. Secara umum, metode yang digunakan terdiri dari tiga tahapan utama, yaitu pemuatan dan pemrosesan data, pembangunan struktur graf, serta pencarian jalur menggunakan BFS.

Pada tahap pertama, data rute penerbangan dibaca dari file routes.csv. Berdasarkan dokumentasi resmi Python [5], modul csv dipilih karena kemampuannya menangani format CSV dengan berbagai variasi pemisah dan encoding secara andal. Data yang tidak valid, seperti kode bandara kosong atau bernilai '\N', disaring dan tidak dimasukkan ke dalam sistem. Berdasarkan dataset OpenFlights [3], setelah proses pemuatan selesai sistem berhasil memuat 67.663 rute dari 3.425 bandara yang tersebar di seluruh dunia.

Pada tahap kedua, seluruh bandara direpresentasikan sebagai node dalam sebuah directed graph yang diimplementasikan menggunakan adjacency list berbasis dictionary. Berdasarkan Cormen et al. [1], adjacency list dipilih karena lebih efisien dalam penggunaan memori untuk graf yang bersifat sparse, yaitu graf dengan jumlah edge yang jauh lebih kecil dibandingkan jumlah edge maksimum yang mungkin. Penggunaan set pada setiap nilai adjacency list juga dipilih untuk mencegah duplikasi tetangga sekaligus mempercepat operasi pencarian.

Pada tahap ketiga, algoritma BFS dijalankan di atas graf yang telah dibangun. Berdasarkan Goodrich et al. [2], implementasi BFS yang efisien mensyaratkan penggunaan struktur data Queue dengan operasi enqueue dan dequeue yang berjalan dalam kompleksitas waktu $O(1)$. Hal ini dipenuhi dengan memanfaatkan collections.deque sebagaimana didokumentasikan oleh Python Software Foundation [4]. Selain pencarian jalur terpendek, sistem juga mendukung analisis jangkauan per level dan pengecekan konektivitas antara dua bandara.

2.2 Flowchart Sistem



Gambar 2.1 Flowchart Sistem

Alur kerja sistem dimulai dari pembacaan file routes.csv menggunakan modul csv Python [5], di mana setiap baris yang valid disimpan ke dalam list rute dan graf adjacency list dibangun dari data tersebut. Selanjutnya, algoritma BFS dijalankan dari bandara asal dengan memasukkan simpul ke dalam queue secara berlapis hingga tujuan ditemukan, dengan kompleksitas waktu $O(V + E)$ [1]. Berdasarkan hasil pengujian menggunakan dataset OpenFlights [3], bandara KZN mampu menjangkau 2.887 bandara dan SIN mampu menjangkau 3.107 bandara dalam maksimal 4 level transit.

2.3 Dependency dan Library yang Digunakan

Program ini dikembangkan menggunakan bahasa Python dan hanya memanfaatkan library standar bawaan Python tanpa memerlukan instalasi paket eksternal tambahan. Berikut adalah dependency yang digunakan.

1. **csv** — Berdasarkan dokumentasi Python Software Foundation [5], modul csv menyediakan fungsionalitas lengkap untuk membaca dan mem-parsing file berformat CSV, termasuk penanganan berbagai variasi pemisah dan encoding.
2. **collections.deque** — Berdasarkan dokumentasi Python Software Foundation [4], deque menyediakan operasi append dan popleft yang keduanya berjalan dalam kompleksitas $O(1)$, menjadikannya pilihan optimal sebagai basis implementasi Queue untuk algoritma BFS. Hal ini juga sejalan dengan Goodrich et al. [2] yang menekankan pentingnya efisiensi operasi antrian dalam implementasi BFS.
3. **typing (Dict, List, Optional, Tuple, Set)** — Berdasarkan dokumentasi Python Software Foundation [4], modul typing digunakan untuk memberikan type hints pada seluruh fungsi dan atribut kelas sehingga kode lebih mudah dibaca dan dipelihara.

2.4 Daftar Fungsionalitas

Sistem yang dibangun memiliki fungsionalitas-fungsionalitas sebagai berikut.

1. **Pemuatan Data (load_data)** — Membaca file routes.csv menggunakan modul csv [5], memvalidasi setiap baris data, dan menyimpan rute-rute valid ke dalam memori. Berdasarkan dataset OpenFlights [3], fungsi ini berhasil memuat 67.663 rute dari 3.425 bandara.
2. **Pembangunan Graf (build_graph)** — Membangun struktur adjacency list dari data rute yang telah dimuat. Berdasarkan Cormen et al. [1], adjacency list merupakan representasi yang tepat untuk graf sparse seperti jaringan penerbangan ini.
3. **Pencarian Rute Terpendek (find_shortest_path)** — Menjalankan algoritma BFS untuk menemukan jalur dengan jumlah transit minimum. Berdasarkan Cormen et al. [1], BFS menjamin jalur yang ditemukan adalah jalur optimal berdasarkan jumlah hop.
4. **Rekonstruksi Jalur (_reconstruct_path)** — Merekonstruksi urutan bandara dari informasi parent yang dikumpulkan selama proses BFS. Berdasarkan Goodrich et al. [2], teknik pelacakan parent merupakan pendekatan standar dalam rekonstruksi jalur pada algoritma BFS.
5. **Pengambilan Detail Penerbangan (get_flight_details)** — Mengambil informasi detail setiap segmen penerbangan pada jalur yang ditemukan, mencakup kode maskapai, ID maskapai, jumlah stops, dan jenis pesawat berdasarkan data dari OpenFlights [3].
6. **Analisis Jangkauan per Level (analyze_reachability_by_level)** — Mengelompokkan bandara yang dapat dijangkau berdasarkan jumlah transit menggunakan BFS berlapis. Berdasarkan hasil pengujian dengan dataset OpenFlights [3], KZN dapat menjangkau 2.887 bandara dan SIN dapat menjangkau 3.107 bandara dalam 4 level transit.
7. **Pengecekan Konektivitas (check_connectivity)** — Memeriksa apakah dua bandara saling terhubung dalam jaringan penerbangan sekaligus mengembalikan jalur terpendek dan jumlah transit. Berdasarkan Cormen et al. [1], pengecekan konektivitas pada graf berarah dapat dilakukan secara efisien menggunakan BFS.

8. **Pencarian Semua Jalur dengan Kedalaman Maksimum**
(find_all_paths_with_max_depth) — Menggunakan pendekatan DFS untuk menemukan semua kemungkinan jalur dengan batasan kedalaman tertentu. Berdasarkan Goodrich et al. [2], pembatasan kedalaman diperlukan untuk mencegah eksplorasi yang tidak terbatas pada graf berukuran besar.

BAB III

HASIL DAN PEMBAHASAN

3.1 Hasil Implementasi Sistem

Berdasarkan hasil implementasi program, sistem berhasil memuat dataset OpenFlights yang terdiri dari 67.663 rute penerbangan dan 3.425 bandara dari berbagai negara di dunia. Seluruh data rute berhasil direpresentasikan ke dalam bentuk directed graph menggunakan adjacency list berbasis dictionary dan set. Setelah proses pembangunan graf selesai, algoritma Breadth-First Search (BFS) berhasil diimplementasikan menggunakan struktur data Queue berbasis collections.deque untuk melakukan pencarian jalur penerbangan dengan jumlah transit minimum.

Program yang dibangun juga berhasil menjalankan seluruh fungsionalitas utama, yaitu pencarian rute terpendek, rekonstruksi jalur, analisis jangkauan per level, pengecekan konektivitas, serta pencarian semua kemungkinan jalur dengan batas kedalaman tertentu. Berdasarkan hasil pengujian, sistem mampu menemukan jalur penerbangan baik untuk rute domestik maupun internasional dengan waktu eksekusi yang cepat meskipun ukuran graf cukup besar.

```
-----  
BFS ROUTE FINDER - Pencarian Rute Terpendek dengan Queue + BFS  
-----  
  
MEMUAT DATA...  
✓ Berhasil memuat 67663 rute dari 3425 bandara  
✓ Graph berhasil dibangun dengan 3425 node  
  
MENYIAPKAN ALGORITMA BFS...
```

Gambar 3.1 Output proses pemuatan data dan pembangunan graph

3.2 Hasil Pengujian Pencarian Rute Terpendek

Pengujian pertama dilakukan pada rute domestik Rusia dari KZN menuju LED. Berdasarkan hasil BFS, sistem berhasil menemukan jalur terpendek dengan jumlah transit minimum. Karena BFS menelusuri graf secara berlapis, jalur pertama yang ditemukan dijamin merupakan jalur dengan jumlah edge paling sedikit sesuai teori BFS oleh Cormen et al. [1].

```

=====
USE CASE 1: KZN -> LED (Rute Domestik Rusia)
=====

ROUTE TERPENDEK DITEMUKAN!

Total Bandara: 2
Total Transit: 0 (tidak termasuk asal dan tujuan)

DETAIL PERJALANAN:
-----
[1] START: KZN
[2] END:   LED

DETAIL PENERBANGAN:
-----

Segmen 1: KZN -> LED
Maskapai: 2B (ID: 410)
Stops: 0
Equipment: CR2

```

Gambar 3.2 Hasil pencarian rute KZN menuju LED

Pengujian kedua dilakukan pada rute Asia Tenggara dari SIN menuju BKK. Sistem berhasil menemukan rute penerbangan langsung tanpa transit. Hal ini menunjukkan bahwa BFS mampu mengidentifikasi direct flight secara optimal ketika tujuan berada pada level pertama pencarian.

```

=====
USE CASE 2: SIN -> BKK (Rute Asia Tenggara)
=====

ROUTE TERPENDEK DITEMUKAN!

Total Bandara: 2
Total Transit: 0 (tidak termasuk asal dan tujuan)

DETAIL PERJALANAN:
-----
[1] START: SIN
[2] END:   BKK

DETAIL PENERBANGAN:
-----

Segmen 1: SIN -> BKK
Maskapai: 3K (ID: 3021)
Stops: 0
Equipment: 320

```

Gambar 3.3 Hasil pencarian rute SIN menuju BKK

Pengujian ketiga dilakukan pada rute internasional Asia dari MNL menuju KIX. Hasil pengujian menunjukkan bahwa sistem dapat menemukan rute internasional dengan jalur optimal berdasarkan jumlah hop minimum. Informasi detail penerbangan seperti maskapai, jumlah stops, dan equipment juga berhasil ditampilkan dari dataset.

Pada pengujian keempat hingga keenam, sistem diuji menggunakan rute yang memerlukan transit, seperti CEK menuju NBC, ACH menuju MUC, dan FLL menuju PNS. Berdasarkan hasil pengujian, BFS berhasil menemukan jalur dengan satu transit yang merupakan jalur optimal pada graf penerbangan tersebut. Hal ini membuktikan bahwa implementasi Queue dan BFS berjalan dengan benar sesuai konsep traversal berlapis.

```

=====
USE CASE 4: CEK -> NBC (Rute dengan 1 Transit di KZN)
=====

RUTE TERPENDEK DITEMUKAN!

Total Bandara: 3
Total Transit: 1 (tidak termasuk asal dan tujuan)

DETAIL PERJALANAN:
-----
[1] START: CEK
[2] TRANSIT: DME
[3] END: NBC

DETAIL PENERBANGAN:
-----

Segmen 1: CEK -> DME
Maskapai: S7 (ID: 4329)
Stops: 0
Equipment: 319

Segmen 2: DME -> NBC
Maskapai: 2B (ID: 410)
Stops: 0
Equipment: CR2

```

Gambar 3.4 Hasil pencarian rute CEK menuju NBC dengan transit

3.3 Analisis Jangkauan Bandara

Sistem juga berhasil melakukan analisis jangkauan bandara berdasarkan level transit menggunakan BFS berlapis. Pengujian dilakukan pada bandara KZN dan SIN dengan maksimal empat level pencarian. Berdasarkan hasil pengujian, bandara KZN mampu menjangkau 2.887 bandara lain dalam empat level transit. Pada level pertama, hanya sejumlah kecil bandara yang dapat dijangkau secara langsung, namun jumlah tersebut meningkat signifikan pada level-level berikutnya akibat efek perluasan traversal BFS pada graf penerbangan global.

```

-----
USE CASE 7: ANALISIS JANGKAUAN PER LEVEL - KZN (Kazan)
-----

Bandara Asal: KZN
Total Bandara Terjangkau: 2887 (tidak termasuk asal)

JANGKAUAN PER LEVEL:
-----

Level 0 (Asal):
-> KZN

Level 1 (Direct Flight / Tanpa Transit):
Jumlah: 21 bandara
-> AER, ALA, ASF, AYT, CEK, DME, DWC, DYU, EGO, FEG, FRU, IST, LBD, LED, OSS ..
. dan 6 lainnya

Level 2 (1 Transit):
Jumlah: 404 bandara
-> EVN, KIV, KJA, KRR, MSQ, OMS, TZX, VKO, AKX, AMS ... dan 394 lainnya

Level 3 (2 Transit):
Jumlah: 1400 bandara
-> BVA, CIA, STN, TSF, IAA, NZH, TGP, THX, KUT, CUN ... dan 1390 lainnya

Level 4 (3 Transit):
Jumlah: 1062 bandara
-> PSR, BRQ, DND, DNR, PDV, PMF, REX, TGZ, TLC, RRG ... dan 1052 lainnya

```

Gambar 3.5 Analisis jangkauan bandara KZN

Sementara itu, bandara SIN menunjukkan konektivitas yang lebih tinggi dibandingkan KZN. Berdasarkan hasil pengujian, SIN mampu menjangkau 3.107 bandara dalam empat level

transit. Hal ini menunjukkan bahwa Singapura merupakan salah satu hub penerbangan internasional dengan tingkat konektivitas global yang sangat tinggi. Hasil analisis ini membuktikan bahwa BFS tidak hanya efektif untuk mencari jalur terpendek, tetapi juga mampu digunakan untuk menganalisis tingkat keterhubungan suatu node dalam graf skala besar.

```
-----
USE CASE 8: ANALISIS JANGKAUAN PER LEVEL - SIN (Singapore)
-----

Bandara Asal: SIN
Total Bandara Terjangkau: 3107 (tidak termasuk asal)

JANGKAUAN PER LEVEL:
-----

Level 0 (Asal):
-> SIN

Level 1 (Direct Flight / Tanpa Transit):
Jumlah: 125 bandara
-> ADL, AKL, AMD, AMS, AUH, BCN, BDO, BKI, BKK, BLR, BNE, BOM, BPN, BWN, CAN .
. dan 110 lainnya

Level 2 (1 Transit):
Jumlah: 1259 bandara
-> ASP, BHQ, CBR, CED, CNS, CPD, KGC, KGI, MGB, MQL ... dan 1249 lainnya

Level 3 (2 Transit):
Jumlah: 1341 bandara
-> ABM, AUU, CUQ, EDR, HID, IRG, KWM, NTN, WEI, ABQ ... dan 1331 lainnya

Level 4 (3 Transit):
Jumlah: 382 bandara
-> ONG, CNM, LAM, ADK, ADQ, AKN, ANI, BET, BRW, CDB ... dan 372 lainnya
```

Gambar 3.6 Analisis jangkauan bandara SIN

3.4 Hasil Pengecekan Konektivitas

Pengujian konektivitas dilakukan untuk memverifikasi apakah dua bandara saling terhubung dalam jaringan penerbangan global. Pada pengujian pertama, sistem berhasil membuktikan bahwa bandara KZN dan LED saling terhubung dengan jalur penerbangan tertentu. Sistem juga menampilkan jumlah transit serta total segmen penerbangan yang diperlukan.

Pada pengujian kedua, dilakukan pengecekan konektivitas antara KZN dan JFK. Berdasarkan hasil pengujian, sistem tidak menemukan jalur yang menghubungkan kedua bandara tersebut dalam graf yang tersedia. Hal ini menunjukkan bahwa BFS mampu mendeteksi kondisi ketika node tujuan tidak dapat dicapai dari node asal.

3.5 Pembahasan Implementasi BFS dan Queue

Implementasi BFS pada proyek ini menggunakan struktur data Queue berbasis collections.deque untuk mendukung operasi enqueue dan dequeue dengan kompleksitas $O(1)$. Penggunaan deque membuat proses traversal graf menjadi efisien meskipun ukuran graf cukup besar. Representasi graf menggunakan adjacency list juga terbukti efektif untuk jaringan

penerbangan yang bersifat sparse. Penggunaan set pada adjacency list membantu mencegah duplikasi edge sekaligus mempercepat proses pencarian tetangga node.

Berdasarkan hasil pengujian seluruh use case, algoritma BFS berhasil memberikan solusi optimal untuk pencarian jalur berdasarkan jumlah transit minimum. Kompleksitas waktu BFS sebesar $O(V + E)$ memungkinkan sistem tetap berjalan efisien meskipun dataset memiliki ribuan node dan puluhan ribu edge. Selain itu, implementasi DFS pada fitur pencarian semua jalur juga melengkapi sistem dengan kemampuan eksplorasi jalur alternatif berdasarkan batas kedalaman tertentu.

BAB IV

KESIMPULAN

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa algoritma Breadth-First Search (BFS) berhasil diterapkan untuk menyelesaikan permasalahan pencarian rute penerbangan terpendek pada jaringan penerbangan global. Representasi graf menggunakan adjacency list mampu menyimpan data jaringan penerbangan secara efisien, sedangkan struktur data Queue berbasis collections.deque mendukung proses traversal BFS dengan performa yang optimal.

Sistem yang dibangun berhasil memuat 67.663 rute penerbangan dan 3.425 bandara dari dataset OpenFlights serta mampu menjalankan berbagai fungsionalitas utama, seperti pencarian jalur terpendek, analisis jangkauan per level, pengecekan konektivitas, dan pencarian jalur alternatif dengan batas kedalaman tertentu. Berdasarkan hasil pengujian, BFS mampu menemukan jalur dengan jumlah transit minimum baik pada rute domestik maupun internasional.

Selain itu, analisis jangkauan menunjukkan bahwa BFS efektif digunakan untuk mengevaluasi tingkat konektivitas suatu bandara terhadap jaringan penerbangan global. Secara keseluruhan, proyek ini membuktikan bahwa kombinasi struktur data Queue dan algoritma BFS sangat sesuai digunakan untuk permasalahan pencarian jalur pada graf berskala besar seperti jaringan penerbangan internasional.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Python*. Hoboken, NJ: John Wiley & Sons, 2013.
- [3] OpenFlights, "Route Database," OpenFlights.org, 2014. [Online]. Available: <https://openflights.org/data.html>. [Accessed: 27 May 2026].
- [4] Python Software Foundation, "collections — Container datatypes," Python 3 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/collections.html>. [Accessed: 27 May 2026].
- [5] Python Software Foundation, "csv — CSV File Reading and Writing," Python 3 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/csv.html>. [Accessed: 27 May 2026].